

How to detect PowerShell abuse techniques in Wazuh for threat hunting.

1- Definition

PowerShell is a powerful tool for administrators, but it is also a favorite weapon for hackers (Hackers use PowerShell to drop fileless malware or use PowerShell to run malicious commands like base64 encoded commands).

2- The four common PowerShell abuse tactics

- **Obfuscation:** Obfuscation means running commands in a hidden way so security tools cannot detect them, like encryption, encoded commands, base64 strings, etc.
- **Download cradles:** Download cradles mean downloading malware from the internet and running it directly in memory, like Invoke-WebRequest, DownloadString, etc.
- **Execution policy bypass:** Execution bypass means bypassing the restrictions on running scripts, like ExecutionPolicy Bypass, ExecutionPolicy Unrestricted, or Exec Bypass., etc.
- **Reconnaissance:** reconnaissance means gathering information about the system, like Whoami, Get-Process, Get-Service, or Get-HotFix.



3- Enable PowerShell logging on to our Windows agents.

The purpose of threat hunting is to detect hidden attacks that normal antivirus software misses.

By default, Windows does not store or does not capture detailed PowerShell logs. We need to enable **script block logging** and **Transcription** via Group Policy

Do: Open the Run dialog, type gpedit.msc, and press **Enter** (or go to Start → Edit Group Policy).

Navigate to:

Computer Configuration → Administrative Templates → Windows Components → Windows PowerShell.

Configure the following policies:

- **Turn on PowerShell Script Block Logging:** Double-click, set to **Enabled**, and click **Apply / OK**.
- **Turn on PowerShell Transcription:** Double-click, set to **Enabled** (specify an output directory if prompted), and click **Apply / OK**.

Open **Command Prompt as Administrator** and force the policy update:

```
gpupdate /force
```

group policy has been updated.

4- Configure Wazuh Agent Log Collection

To ensure the Wazuh manager receives these logs from your Windows endpoints:

Do: Navigate to your Wazuh configuration or agent config group for Windows machines → Add the appropriate log collection block to ingest Windows PowerShell operational and analytical event logs.

```
<localfile>  
  <log_format>eventchannel</log_format>  
  <location>Microsoft-Windows-PowerShell/Operational</location>  
</localfile>
```

→ Save and restart/apply the configuration changes.

The purpose of this code block is to inform the Wazuh agent or Windows machine that it needs to send or pass PowerShell logs to the Wazuh manager.

5- Define Custom Wazuh Detection Rules

Now we need to define rules on our Wazuh manager that will detect encoded commands or suspicious keywords.

Access the custom rules directory on the manager to define our rules

Do: In `/var/ossec/etc/rules/ local_rules.xml` past this:

```
<group name="powershell, threat_hunting,">  
  <!-- Rule 1: Detect Base64 Encoded Commands (Obfuscation) -->  
  <rule id="100601" level="10">  
    <if_sid>91802</if_sid> <!-- Checks PowerShell Operational Logs -->  
    <field name="win.eventdata.scriptBlockText" type="pcre2">(?!)(-enc|-encodedcommand|-  
e\s)</field>  
    <description>PowerShell Abuse: Encoded Command Detected (Potential Obfuscation)</description>  
    <mitre>
```

```

        <id>T1027</id> <!-- Obfuscated Files or Information -->
    </mitre>
</rule>

<!-- Rule 2: Detect Download Cradles (Malware Downloading) -->
<rule id="100602" level="12">
    <if_sid>91802</if_sid>
    <field name="win.eventdata.scriptBlockText"
type="pcre2">(?!)(Net\.WebClient|DownloadString|Invoke-WebRequest)</field>
    <description>PowerShell Abuse: Suspicious Download Command Detected</description>
    <mitre>
        <id>T1059.001</id> <!-- PowerShell -->
    </mitre>
</rule>
<rule id="100603" level="7">
<if_sid>91802</if_sid>
    <field name="win.eventdata.scriptBlockText" type="pcre2">(?!)-windowstyle\s+hidden|-
executionpolicy\s+bypass</field>
    <description>PowerShell Abuse: Execution Policy Bypass or Hidden Window</description>
</rule>
</group>

```

Basically:

Rule 1 will detect base64 encoded commands,

rule 2 will detect download cradles or such commands,

Save the file and restart the Wazuh manager service to load the new rules.

```
sudo systemctl restart wazuh-manager
```

6- Testing & Verification

Execute Test Commands on the Windows Endpoint

Do: go to your Windows machine, open PowerShell as administrator, and:

A- For Base64 Encoded Command Test: Run an encoded command in PowerShell

```
$encoded = [System.Convert]::ToBase64String([System.Text.Encoding]::Unicode.GetBytes("Write-Host
"Hello, Wazuh"))
powershell.exe -EncodedCommand $encoded
```

This encoded command will simply print "Hello, Wazuh"

B-For Download Cradle Test: Run a test download command pulling a harmless file into a local directory
C:\Temp:

```
# Step 1: Create the target directory if it doesn't exist
```

```
New-Item -ItemType Directory -Force -Path "C:\Temp"
```

Step 2: Run the download cradle command

```
Invoke-WebRequest -Uri "https://github.com/octokit/octokit.rb/archive/refs/heads/master.zip" -OutFile "C:\Temp\master.zip"
```

It is downloading this GitHub file into a temp folder inside our C drive

7- Validate Alerts in the Wazuh Dashboard

Now we will go to our Wazuh SIEM.

Do: Open the **Wazuh Dashboard** and navigate to **Modules** → **Security events** or check general alerts to see the triggered rule for encoded or suspicious PowerShell execution. There we can see that it has shown us that the encoded or suspicious command was run or executed. We can also see the command we executed or the file we downloaded. And in this way, we can also see the encoded command that was run, which printed "Hello, Wazuh".

Another way to see it:

Do: select Medium severity (**Discover**) to inspect raw event data

Let's perform some filter:

Do: Select your Windows host index/pattern (agent.name). Add a filter rule for description matching your alert signatures (rule.description). Add a filter or column for script_block_text to view the exact executed command payload and verify visibility into the hidden or encoded strings(data.win.eventdata.scriptBlockText).

